

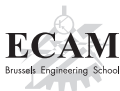
Artificial Intelligence – Machine Learning

Ruben Hillewaere

`hil@ecam.be`

ECAM
Institut supérieur industriel

Feb 2025



1. Introduction
2. Machine learning background
3. Regression
 - Linear regression
 - Polynomial regression
4. Classification

Stating the classification problem

Standard ML Classifiers

- Naive Bayes model
- Nearest neighbors
- Logistic regression
- Linear discriminant functions
- The perceptron algorithm
- Neural networks
- Linear basis functions
- Support vector machines
- Decision trees
- Random forests

Boosting

Model evaluation
Feature selection

5. Unsupervised Learning

K-means clustering
Hierarchical clustering
Principal component analysis

6. Reinforcement Learning

Stating the reinforcement problem
Finite Markov decision processes

7. Sequential data

Markov models
Hidden Markov models

8. Sustainable AI: environmental impact

9. Bibliography

1. Introduction

What does an engineer need machine learning for?

Let's brainstorm!

What does an engineer need machine learning for?

Let's brainstorm!

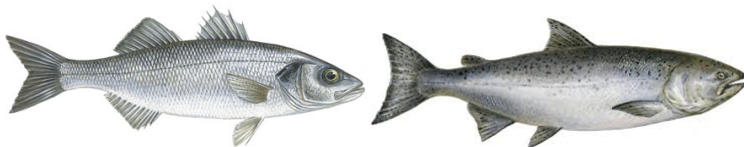
Non-exhaustive list of engineering application fields:

- ▶ Maintenance, default detection
- ▶ Logistics, scheduling, supply chains
- ▶ Cybersecurity, spamfilter, virus detection
- ▶ Prediction, e.g.: next word, density distribution, etc.
- ▶ Classification: signal types, automatic tagging, etc.
- ▶ Recommendation: social media, marketing, music/film, etc.
- ▶ Image correction
- ▶ Smart house
- ▶ ...

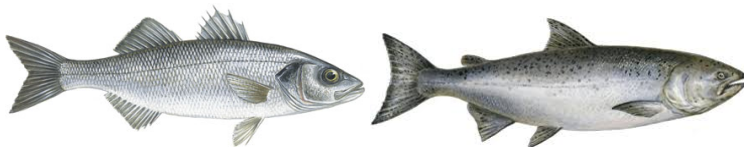
More general application fields

- ▶ Natural Language Processing
- ▶ Search machines
- ▶ Medical diagnosis
- ▶ Drug discovery
- ▶ Bioinformatics and chemoinformatics
- ▶ Classification of DNA sequences
- ▶ Speech and handwriting recognition
- ▶ Object recognition in computer images
- ▶ Game playing
- ▶ Robot movement
- ▶ ...

A fish classification example

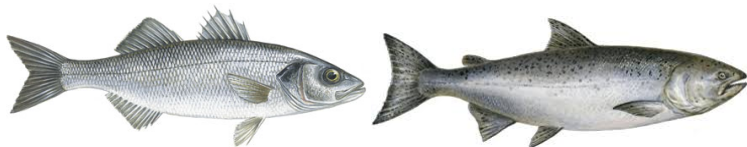


A fish classification example



Seabass versus Salmon:

A fish classification example

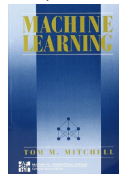


Seabass versus Salmon:

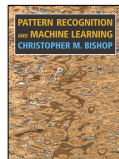
- ▶ Measure fish “features”:
 x_1 = length, x_2 = width, x_3 = weight, x_4 = area of fins, x_5 = mean spectral reflectance, x_6 = ...
- ▶ Generalization: based on the feature vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$, determine the “target class label” $y = 0$ (sea bass) or $y = 1$ (salmon)
- ▶ You need prior knowledge: a database of correctly labeled fish feature vectors verified by a fish expert

References: books

- ▶ T. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997



- ▶ C. M. Bishop. *Pattern recognition and machine learning*, volume 4. Springer New York, USA, 2006



2. Machine learning background

What is Machine Learning?

Several definitions:

- ▶ Machine learning is a branch of Artificial Intelligence in which algorithms and techniques are developed to enable the computer to “learn”.
- ▶ ML is the automatic extraction of information (rules, patterns) from data.
- ▶ ML is directly related to data mining, statistics, pattern recognition, information retrieval and theoretical informatics.
- ▶ ...

What is the difference between AI and ML?

Artificial Intelligence is the broader concept of machines being able to carry out tasks in a way that we would consider “smart”.

And,

Machine Learning is a current application of AI based around the idea that we should really just be able to give machines access to data and let them learn for themselves.

What exactly is Learning?

- ▶ “Learning is the human process that allows us to acquire the skills necessary to adapt to the multitude of situations we encounter.” [Japkowicz and Shah (2011)]

What exactly is Learning?

- ▶ “Learning is the human process that allows us to acquire the skills necessary to adapt to the multitude of situations we encounter.” [Japkowicz and Shah (2011)]
- ▶ In Machine Learning terms, we say:
“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .” [Mitchell (1997)]

What exactly is Learning?

- ▶ “Learning is the human process that allows us to acquire the skills necessary to adapt to the multitude of situations we encounter.” [Japkowicz and Shah (2011)]
- ▶ In Machine Learning terms, we say:
“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .” [Mitchell (1997)]

In the fish example:

- ▶ Experience E : correctly labeled fish data
- ▶ Task T : is the next fish sea bass or salmon?
- ▶ Performance measure P : the percentage of correct predictions (for example!)

Stages in a Machine Learning process

- ▶ Gather a training set of data instances, represented by feature vectors $\mathbf{x}$



- ▶ Pre-processing step: feature extraction
- ▶ Training or learning phase: tuning of parameters of an adaptive model, resulting in a function $y(\mathbf{x})$ or $\mathbf{y}(\mathbf{x})$
- ▶ Testing phase: use trained model to make decisions for instances of an unknown test set, i.e. generalization

Types of Learning

- ▶ Supervised learning
 - ▶ Regression
 - ▶ Classification
- ▶ Unsupervised learning
 - ▶ Principal component analysis
 - ▶ Clustering
 - ▶ Density estimation
 - ▶ ...
- ▶ Reinforcement learning
- ▶ (Semi-supervised learning: subgroup discovery)

Supervised Learning

- ▶ Training data is a set of N input vectors $\mathbf{x}^i$ with their respective target labels t^i , where $1 \leq i \leq N$.
- ▶ When $t^i \in \mathbb{R}$, the task is called *regression*.
When $t^i \in \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$, the task is called *classification*.
- ▶ With matrix notation:

$$\mathbf{X} = \begin{pmatrix} x_1^1 & \cdots & x_n^1 \\ \vdots & \ddots & \vdots \\ x_1^N & \cdots & x_n^N \end{pmatrix} \quad \text{and} \quad \mathbf{t} = \begin{pmatrix} t^1 \\ \vdots \\ t^N \end{pmatrix}$$

in which N is the number of training instances and n is the number of input features.

- ▶ Note that many books will write the input vectors as $\mathbf{x}_1, \dots, \mathbf{x}_N$, in which every $\mathbf{x}_i$ is an n -dimensional input vector.

3. Regression

Regression

- ▶ Prediction of a continuous target variable t based on an input vector $\mathbf{x}$
- ▶ Method: we construct a function $y(\mathbf{x})$ whose values for new inputs $\mathbf{x}$ are the predictions for the corresponding values of t
- ▶ Underlying idea: we model the distribution $p(t|\mathbf{x})$ that expresses the uncertainty about the value of t for each $\mathbf{x}$
- ▶ Predictions are made by minimizing the expected value of a suitable loss function

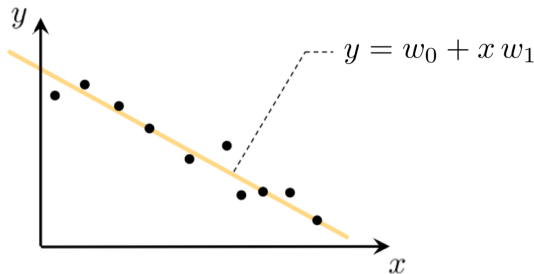
Linear regression in 2D

- ▶ Linear function:

$$y(x, \mathbf{w}) = w_0 + w_1 x$$

is fitting the training data

- ▶ $\mathbf{w}$ is the *weight vector* : w_0 is the *bias* and w_1 is the slope

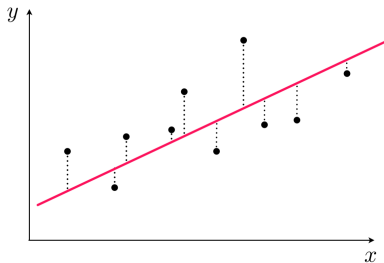


Linear regression in 2D

- Fitting is minimizing the *least squares* error function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N [y(x_i, \mathbf{w}) - t_i]^2$$

where we sum over the N data points (x_i, t_i) of the training set

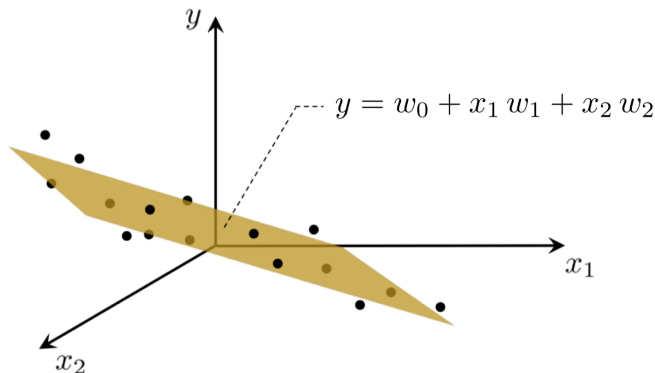


Linear regression in 3D

- Linear function:

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1 x_1 + w_2 x_2$$

is fitting the training data



Linear regression

- ▶ In general, the linear function is:

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = w_0 + \sum_{j=1}^n w_jx_j$$

- ▶ Notation trick: if we denote $x_0 = 1$, we can write:

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^n w_jx_j = \mathbf{w}^T \tilde{\mathbf{x}}$$

where

$$\mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix} \quad \text{and} \quad \tilde{\mathbf{x}} = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}$$

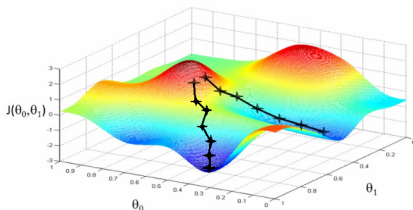
Linear regression: learning

- ▶ Learning is minimizing the *loss* function or *cost* function:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N [y(\mathbf{x}^i, \mathbf{w}) - t^i]^2 = \frac{1}{2} \sum_{i=1}^N [\mathbf{w}^T \tilde{\mathbf{x}}^i - t^i]^2$$

where we sum over the N data points $(\tilde{\mathbf{x}}^i, t^i)$ of the training set

- ▶ There exist many optimization algorithms to find the optimal weights $\mathbf{w}$, a well-known one is the *gradient descent*



Polynomial regression

- ▶ Polynome:

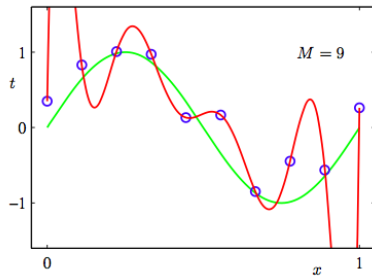
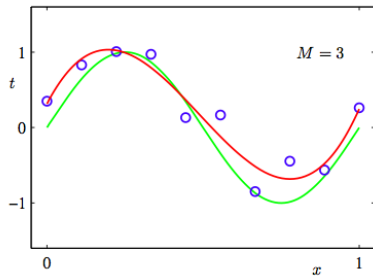
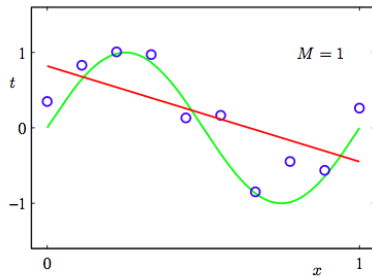
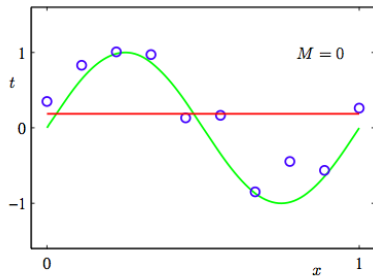
$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

- ▶ Fitting is (again!) minimizing the error function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N [y(x_i, \mathbf{w}) - t_i]^2$$

- ▶ The degree M of the polynomial introduces the concept of *model selection*: we are looking for a model that achieves good *generalization*. Be careful for *overfitting* !!

Polynomial regression



Linear basis function models

- ▶ The *linear regression* model can be extended for addressing non-linearity:

$$y(\mathbf{x}, \mathbf{w}) = \mathbf{w}^T \widetilde{\phi(\mathbf{x})} = \sum_{j=0}^M w_j \phi_j(\mathbf{x})$$

where $\phi_j(\mathbf{x})$ are fixed nonlinear basis functions, and $\phi_0(\mathbf{x}) = 1$

- ▶ E.g., with a single input variable, if $\phi_j(x) = x^j$, we obtain polynomial regression.
- ▶ Other well-known non-linear basis functions are:
 - ▶ Gaussian : $\phi_j(x) = \exp \left\{ -\frac{(x-\mu_j)^2}{2s^2} \right\}$
 - ▶ Logistic sigmoid: $\phi_j(x) = \sigma \left(\frac{x-\mu_j}{s} \right)$ with $\sigma(a) = \frac{1}{1+\exp(-a)}$
 - ▶ Tangent hyperbolic, Fourier basis, wavelets, ...

Linear basis function models

- Generally, the error function also contains a *regularization* term to avoid having too large weight vectors, for example:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N \left[\mathbf{w}^T \phi(\mathbf{x}_i) - t_i \right]^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$$

where λ is the regularization parameter.

4. Classification

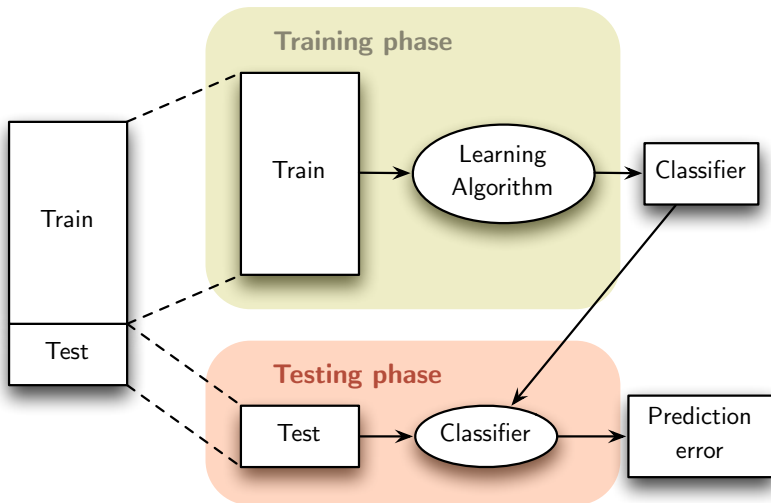
Stating the classification problem

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ." [Mitchell (1997)]

Classification:

- ▶ Experience E : training data $\mathbf{x}^i$ with target labels $t^i \in \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$
- ▶ Task T : classification of unlabeled test instances
- ▶ Performance measure P , e.g. the fraction of correctly classified test instances

Training and testing



Different approaches for classification problems

1. Find a function $f(\mathbf{x})$ that maps an input instance $\mathbf{x}$ directly onto a target value in $\{\mathcal{C}_1, \dots, \mathcal{C}_K\}$. This is called a *discriminant function*.
2. Break the problem down in two stages:
 - ▶ *Inference stage*: determination of the joint probability distribution $p(\mathbf{x}, \mathcal{C}_k)$
 - ▶ *Decision stage*: make optimal class assignments based on that probability distribution

Inference

Terminology:

- ▶ $p(\mathbf{x}, \mathcal{C}_k)$: joint probability distribution
- ▶ $p(\mathcal{C}_k|\mathbf{x})$: posterior class probabilities
- ▶ $p(\mathbf{x}|\mathcal{C}_k)$: class-conditional densities
- ▶ $p(\mathcal{C}_k)$: prior class probabilities

These distributions are related by Bayes' theorem:

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})}$$

where

$$p(\mathbf{x}) = \sum_k p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)$$

Modelling these distributions leads to a *generative classification model*.

Standard machine learning classifiers

Well-known classifiers for data points $\mathbf{x} = (x_1, \dots, x_n)$:

1. **Naive Bayes model:** benchmark, generative model
2. **Nearest neighbors (K -NN):** popular, easy to understand
3. **Logistic regression:** probabilistic discriminative model
4. **Neural networks (NN):** basis for deep learning
5. **Support vector machines (SVM):** popular top classifier
6. **Decision trees:** basic classifier
7. **Random forests:** top classifier, ensemble of decision trees

Naive Bayes model

- Models the posterior class probabilities:

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})}$$

- Naive independence assumption:

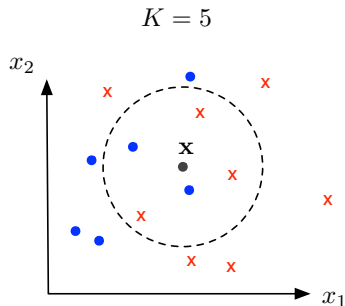
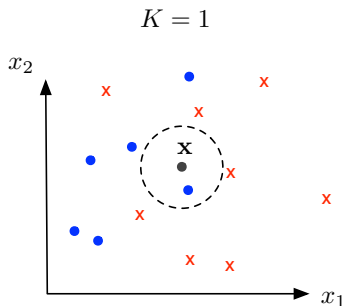
$$p(\mathbf{x}|\mathcal{C}_k) = p(x_1, \dots, x_n|\mathcal{C}_k) = \prod_i p(x_i|\mathcal{C}_k)$$

- Solves the expression:

$$\operatorname{argmax}_{\mathcal{C}_k} p(\mathcal{C}_k) \prod_i p(x_i|\mathcal{C}_k)$$

Nearest neighbors (K -NN)

- ▶ Instance-based learning method
- ▶ Requires a distance or a similarity measure
- ▶ Assigns test instance $\mathbf{x}$ to the majority class of the K nearest training instances



Nearest neighbors (K -NN)

To compute the underlying probability distributions, we denote:

N : total # of data points, N_k : # of data points in class $\mathcal{C}_k$,

K : number of neighbors in the sphere around test instance $\mathbf{x}$,

K_k : number of neighbors belonging to class $\mathcal{C}_k$,

V : volume of the sphere

Then:

$$p(\mathbf{x}|\mathcal{C}_k) = \frac{K_k}{N_k V} \quad ; \quad p(\mathbf{x}) = \frac{K}{NV} \quad ; \quad p(\mathcal{C}_k) = \frac{N_k}{N}$$

So,

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})} = \frac{K_k}{K}$$

Logistic regression

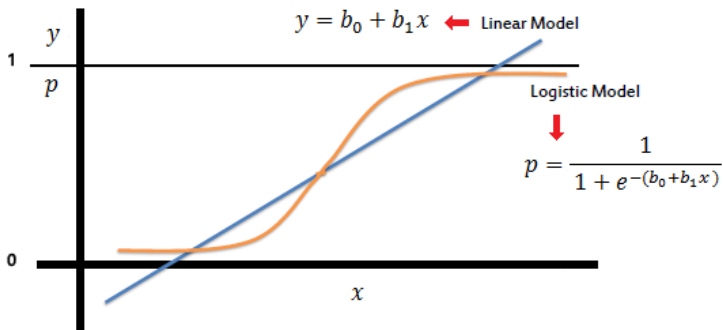
- ▶ Binary classification: classes $\mathcal{C}_1$ and $\mathcal{C}_2$ with target labels $t = 0$ and $t = 1$
- ▶ Computes the output $y = \sigma(\mathbf{w}^T \tilde{\mathbf{x}})$, where σ is the *logistic sigmoid* function:

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

- ▶ $\sigma(a) \rightarrow 0$ for $a \rightarrow -\infty$ and $\sigma(a) \rightarrow 1$ for $a \rightarrow +\infty$
- ▶ y can be interpreted as a probability
- ▶ x is assigned to $\mathcal{C}_1$ if $y < 0.5$, and to $\mathcal{C}_2$ if $y > 0.5$
- ▶ Typically the error function is *cross entropy cost*:

$$E(\mathbf{w}) = - \sum_{i=1}^N [t_i \ln y_i + (1 - t_i) \ln(1 - y_i)]$$

Logistic regression



Linear discriminant functions

Definition

Linear discriminant functions are dividing the input space into decision regions whose boundaries are linear functions of the input vector $\mathbf{x}$. In other words, the *decision boundaries*, also called *decision surfaces*, are $(D - 1)$ -dimensional hyperplanes within the D -dimensional input space.

Overview:

- ▶ Two classes
- ▶ Multiple classes
- ▶ Fisher's linear discriminant
- ▶ Perceptron algorithm

Linear discriminant functions for two classes

Recall from earlier courses:

- ▶ The equation of a plane is $ax + by + cz + d = 0$, where (a, b, c) is a normal vector of the plane.
- ▶ The perpendicular distance δ between a point (x_1, y_1, z_1) and a plane $ax + by + cz + d = 0$ is given by

$$\delta = \frac{|ax_1 + by_1 + cz_1 + d|}{\sqrt{a^2 + b^2 + c^2}}$$

- ▶ In matrix notation, a $(D - 1)$ -dimensional hyperplane is written as

$$\mathbf{w}^T \tilde{\mathbf{x}} = 0$$

by denoting $x_0 = 1$.

Linear discriminant functions for two classes

- ▶ The simplest linear discriminant function of the input vector is

$$y(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x} = \mathbf{w}^T \tilde{\mathbf{x}}$$

where $\mathbf{w}$ is called a *weight vector*.

- ▶ The decision boundary $y(\mathbf{x}) = 0$ is a $(D - 1)$ -dimensional hyperplane within the D -dimensional input space
- ▶ If $y(\mathbf{x}) \geq 0$, the input vector $\mathbf{x}$ is assigned to class $\mathcal{C}_1$.
If $y(\mathbf{x}) < 0$, the input vector $\mathbf{x}$ is assigned to class $\mathcal{C}_2$.

Linear discriminant functions for two classes

Properties:

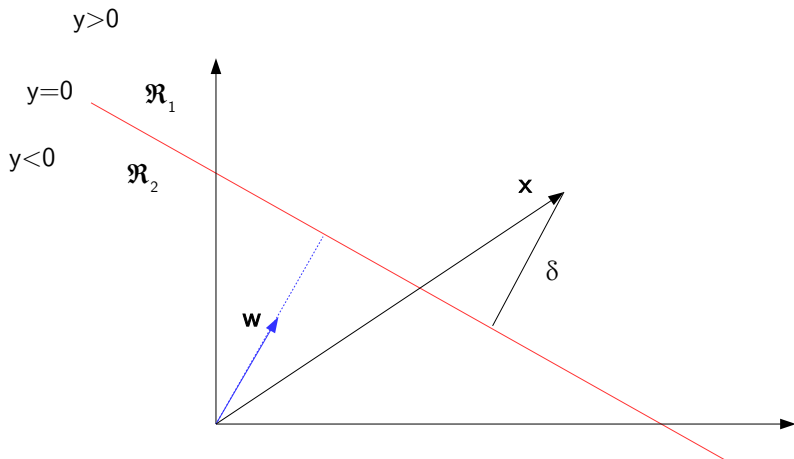
- ▶ $\mathbf{w}$ is a normal vector of the hyperplane. For any two points $\mathbf{x}_1$ and $\mathbf{x}_2$ on the hyperplane:

$$y(\mathbf{x}_1) - y(\mathbf{x}_2) = \mathbf{w}^T (\tilde{\mathbf{x}}_1 - \tilde{\mathbf{x}}_2) = 0$$

- ▶ $y(\mathbf{x})$ is proportional to the signed distance between the input vector $\mathbf{x}$ and the hyperplane:

$$\delta = \frac{|y(\mathbf{x})|}{\|\mathbf{w}\|}$$

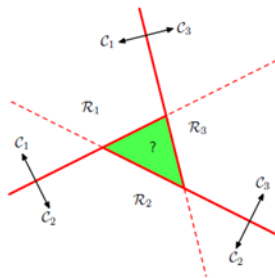
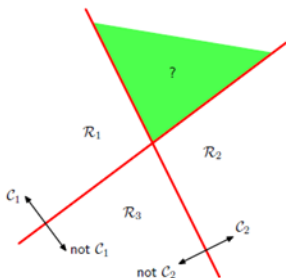
Linear discriminant functions for two classes



Linear discriminant functions for multiple classes

Combining two-class discriminants leads to ambiguous regions!

- ▶ *one-versus-the-rest* classifier
- ▶ *one-versus-one* classifier



Linear discriminant functions for multiple classes

Possible solution: build a single K -class discriminant of K linear functions

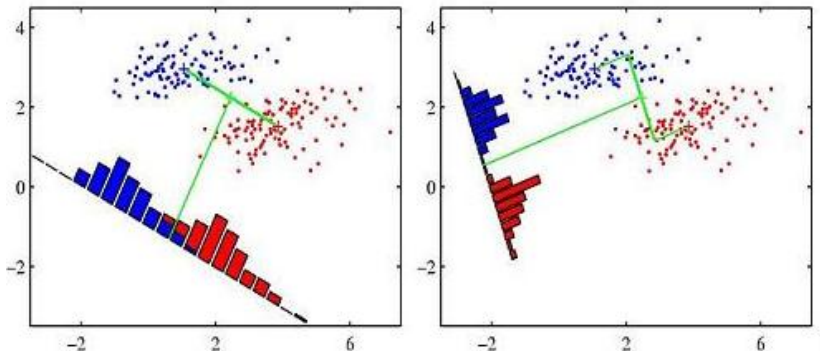
$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x}$$

An instance $\mathbf{x}$ is assigned to class $\mathcal{C}_k$ if $y_k(\mathbf{x}) > y_j(\mathbf{x})$ for all $j \neq k$.

Fisher's linear discriminant for two classes

Determine weight vector $\mathbf{w}$ to obtain:

- ▶ maximal separation between the projected class means
- ▶ minimal variance within each class



The perceptron algorithm

Given two classes $\mathcal{C}_1$ and $\mathcal{C}_2$ with target labels $t = +1$ resp. $t = -1$, the *perceptron* algorithm constructs a generalized linear model of the form

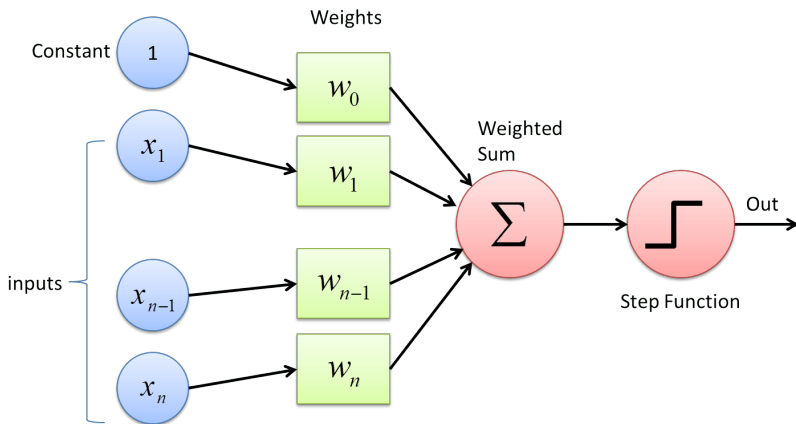
$$y(\mathbf{x}) = f\left(\mathbf{w}^T \tilde{\mathbf{x}}\right)$$

where f is a nonlinear *activation function*:

$$f(a) = \begin{cases} +1, & a \geq 0 \\ -1, & a < 0. \end{cases}$$

So, the instance $\mathbf{x}$ is assigned to $\mathcal{C}_1$ if $\mathbf{w}^T \tilde{\mathbf{x}} > 0$, and to $\mathcal{C}_2$ otherwise.

The perceptron algorithm



The perceptron algorithm

Learning through iteration:

- ▶ Start with random (small) weights
- ▶ For each training instance $\mathbf{x}$, update the weight vector:

$$w_i \leftarrow w_i + \eta(t - y(\mathbf{x}))x_i$$

where $\eta > 0$ is the *learning rate* and $t = \pm 1$ is the target label.

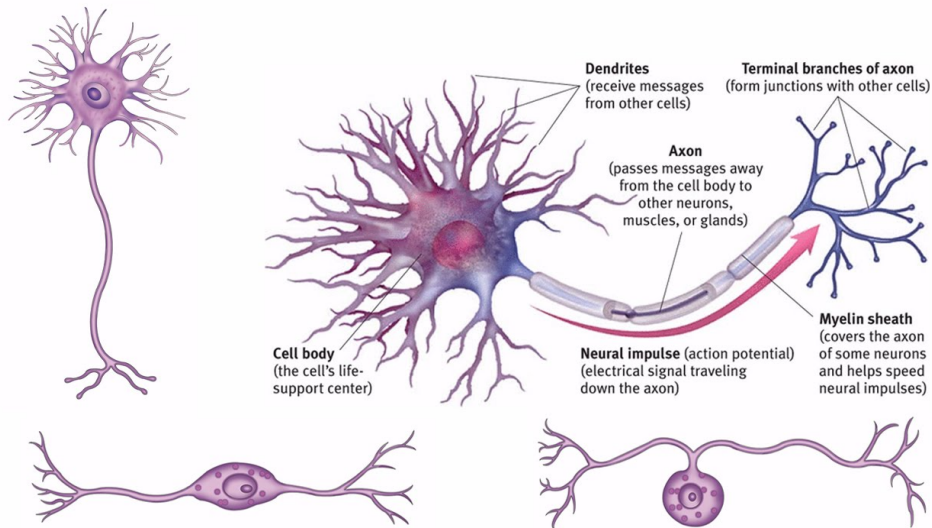
Notice:

- ▶ If $\mathbf{x}$ is correctly classified, nothing happens.
- ▶ What happens if $y(\mathbf{x}) = -1$ when the actual target is $t = +1$? And what if $y(\mathbf{x}) = +1$ while $t = -1$?
- ▶ A previously correctly classified instance can become misclassified! What would you do about that?

Neural networks

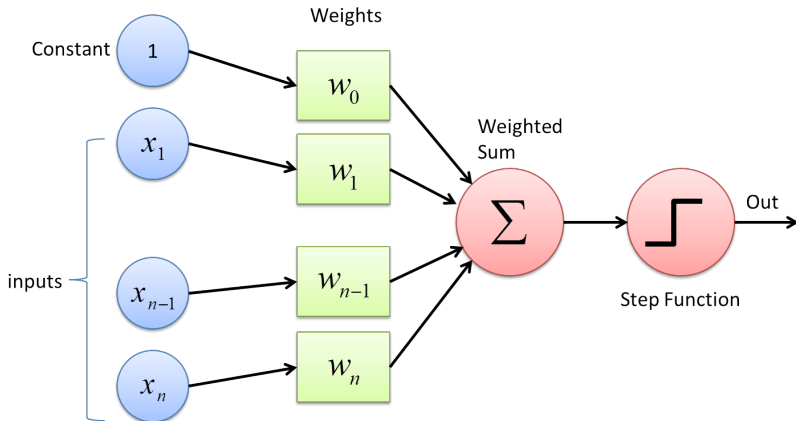
- ▶ “Old” idea that has only recently been truly effective, especially on image classification [Krizhevsky et al. (2012)]
- ▶ Originally based on analogies with biological systems
- ▶ Kind of extension of the perceptron algorithm, also called the *multilayer* perceptron
- ▶ Aims to address limitations of working with fixed linear basis functions

Neural networks: what's a neuron?



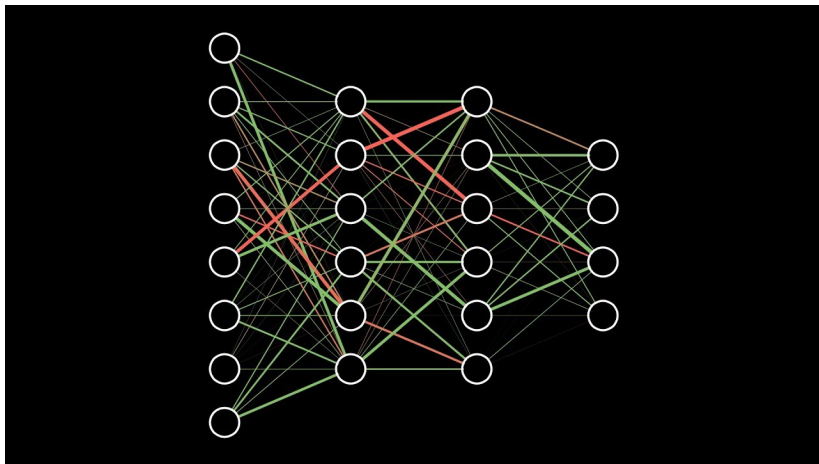
Neural networks

One neuron can be modeled by ... the perceptron!



Neural networks

Our brain can be modeled by ... a neural network ;-)



Neural networks

A neural network is a function $\mathbf{y} = f(\mathbf{x}, \mathbf{w})$ of the input vector and (lots of) weights and biases.

Training a neural network comes down to adjusting those weights in order to make the output $\mathbf{y}$ of the network “fit” to the desired output, the target vector $\mathbf{t}$, so actually a neural network is a function approximator:

$$\mathbf{y}(\mathbf{x}, \mathbf{w}) \approx \mathbf{t}(\mathbf{x})$$

For one training instance, we want to maximize the performance

$$P(\mathbf{w}) = -\|\mathbf{t} - \mathbf{y}\|^2$$

or equivalently, to minimize the cost $C(\mathbf{w}) = \|\mathbf{t} - \mathbf{y}\|^2$.

Neural networks

Therefore, we need to change the discontinuous activation function, such as the *logistic sigmoid* (again!):

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

Properties:

- ▶ $\sigma(x) \rightarrow 0$ for $x \rightarrow -\infty$ and $\sigma(x) \rightarrow 1$ for $x \rightarrow +\infty$ (again!)
- ▶ $\sigma(x)$ is expressing how “active” the neuron is
- ▶ $\sigma'(x) = (1 - \sigma)\sigma$

Neural networks

- So, the activation of a neuron receiving input $\mathbf{x}$ will look like

$$\sigma \left(\sum_{j=0}^n w_j x_j \right) = \sigma \left(w_0 + \sum_{j=1}^n w_j x_j \right)$$

which shows the bias w_0 becomes meaningful as a threshold for inactivity.

- Remark: currently, the “ReLU”-function is frequently used instead of the sigmoid function:

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Neural networks

- ▶ Very nice video explaining the layered structure of a neural network: <http://youtu.be/aircAruvnKk>
- ▶ Simplest neural network: 2 neurons in line (example on the blackboard)
- ▶ See this network at work in this video:
<http://youtu.be/uXt8qF2Zzfo?t=39m36s>
- ▶ Small neural network (example on the blackboard)

Neural networks

We touched upon some important words here:

- ▶ Backpropagation
- ▶ Gradient descent
- ▶ Mini-batches and stochastic gradient descent

Note: a NN can recognize a digit, but he still can't draw one!

Linear basis functions

The linear function $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ can be extended to

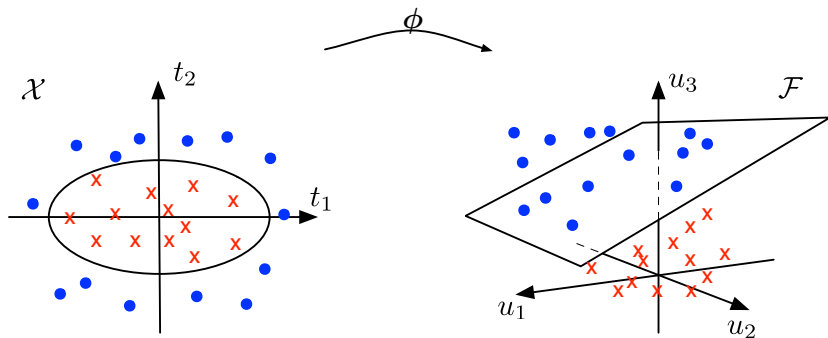
$$y(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x})$$

where $\phi_j(\mathbf{x})$ are fixed nonlinear functions, called the *basis functions*. The vector $\phi(\mathbf{x})$ is called the *feature vector*.

Note:

- ▶ Recall that $\phi_0(\mathbf{x}) = 1$ for notation reasons.
- ▶ this is still called a linear model, because $y(\mathbf{x}, \mathbf{w})$ is linear with respect to the weight vector $\mathbf{w}$.

Linear basis functions



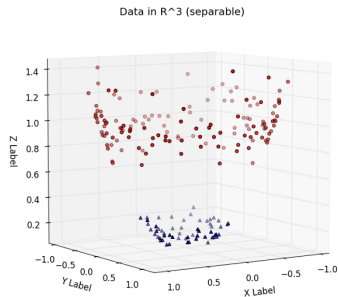
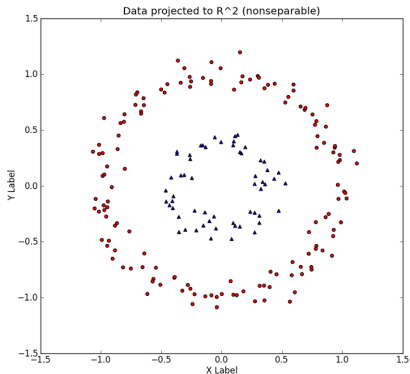
$\mathcal{X}$ is called the *input space*, $\mathcal{F}$ is called the *feature space*.

Linear basis functions: an example

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3 : \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \mapsto \begin{pmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{pmatrix}$$

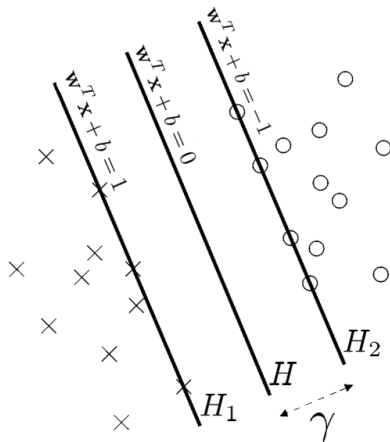
Linear basis functions: an example

$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3 : \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \mapsto \begin{pmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{pmatrix}$$



Support vector machines (SVM)

- ▶ Introduced by Vapnik (1995)
- ▶ Based on the linear perceptron for binary classification
- ▶ Learns a maximal-margin boundary between classes:



Support vector machines: the margin

- ▶ An input vector $\mathbf{x}_i$ with target label $t_i = \pm 1$ is correctly classified if

$$t_i (\mathbf{w}^T \mathbf{x}_i + b) > 0,$$

else it is not.

- ▶ The distance between a well-classified input vector $\mathbf{x}_i$ and the hyperplane H is equal to

$$\frac{|y(\mathbf{x}_i)|}{\|\mathbf{w}\|} = \frac{t_i (\mathbf{w}^T \mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

- ▶ The *margin* γ is the smallest distance between the decision boundary and any of the training samples, i.e. the distance between the closest point and the hyperplane H .

Support vector machines: the margin

- ▶ Say $\mathbf{x}_n$ is the closest point from the training set, then we can scale $\mathbf{w}$ and b to set $t_i (\mathbf{w}^T \mathbf{x}_n + b) = 1$, such that the margin is equal to

$$\gamma = \frac{1}{\|\mathbf{w}\|}$$

- ▶ In order to maximize the margin γ , we need to minimize $\|\mathbf{w}\|$ under the constraint

$$t_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

meaning all training instances have to be correctly classified. This is done with a technique called *Lagrange multipliers*.

- ▶ The resulting decision boundary only depends on a small number of data instances called the *support vectors*.

Support vector machines: the weight vector

- The update of the weight vector $\mathbf{w}$ in the perceptron algorithm is achieved by:¹

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} + \eta t_i \mathbf{x}_i$$

Therefore, $\mathbf{w}$ is essentially a linear combination of the training data points $\mathbf{x}_i$:

$$\mathbf{w} = \sum_i \alpha_i t_i \mathbf{x}_i$$

- The *dual* form of the separating hyperplane $H : \mathbf{w}^T \mathbf{x} + b = 0$ is given by

$$\sum_i \alpha_i t_i (\mathbf{x}_i^T \mathbf{x}) + b = 0$$

¹This formula is not identical to the one seen earlier, but the main idea remains unchanged.

Support vector machines: hyperplanes in the feature space

- ▶ In the feature space $\mathcal{F}$, the equation of the separating hyperplane is

$$\mathbf{w}^T \phi(\mathbf{x}) + b = 0$$

and its dual form is

$$\sum_i \alpha_i t_i \left(\phi(\mathbf{x}_i)^T \phi(\mathbf{x}) \right) + b = 0$$

- ▶ Problem: computing those inner products in the high-dimensional feature space is infeasible!
- ▶ Solution by the so-called *Kernel trick*: computing the inner products in the feature space directly as a function of the original input points!

Support vector machines: kernel functions

- ▶ Example on the blackboard:

$$\phi : \mathcal{X} \rightarrow \mathcal{F} : (t_1, t_2) \mapsto (t_1^2, \sqrt{2}t_1t_2, t_2^2)$$

- ▶ A *Kernel* is a symmetric, positive definite function that expresses the similarity between two instances, e.g. the *radial basis function*:

$$K_{RBF}(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

- ▶ Kernel trick: $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$
- ▶ Defining a kernel implicitly defines the feature space $\mathcal{F}$ and the nonlinear mapping ϕ !!
- ▶ SVM illustrated: <http://youtu.be/N1v0golbjSc?t=1152>

Support vector machines: hard and soft margin

- ▶ Non-linearly separable problems can also be solved by allowing misclassifications at a certain cost.
- ▶ We define the *slack variables* ξ_i :

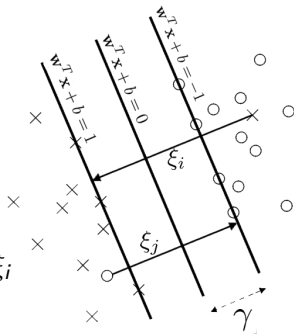
$$\xi_i = \max(0, \gamma - t_i(\mathbf{w}^T \mathbf{x}_i + b))$$

- ▶ We want to minimize

$$\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i$$

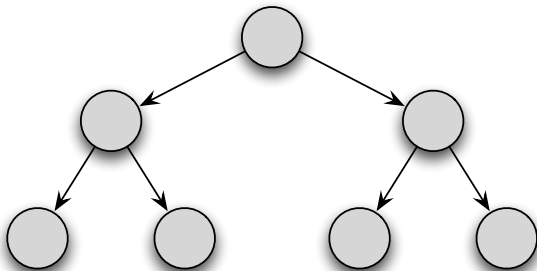
under the constraint

$$t_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$$



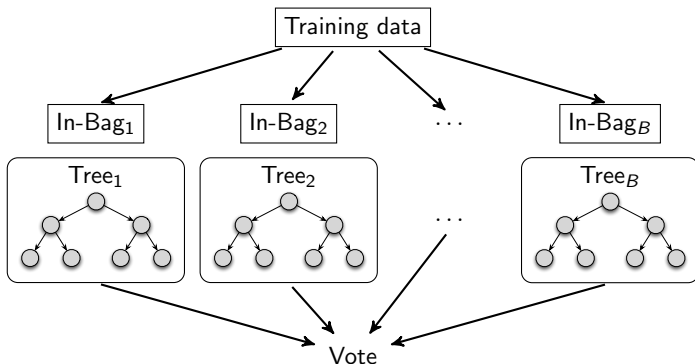
Decision trees

- ▶ Tree: hierarchy of branches and nodes
- ▶ Sequential decision making process by traversing the tree



Random forests [Breiman, 2001]

- ▶ Ensemble of decision trees, majority vote
- ▶ Key idea: each base learner is a weak learner, but when they are aggregated they can form a strong learner.

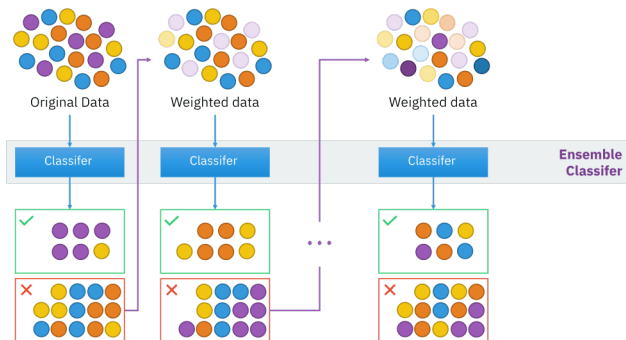


Random forests

- ▶ Bootstrap sampling: training data is sampled with replacement (In-Bag)
- ▶ Only a small random subset of features is used at every split point (parameter m)
- ▶ Internally keeps track of performance using the left-out instances (Out-Of-Bag error)
- ▶ Able to deal with high-dimensional data and irrelevant features
- ▶ Random forest is an example of an ensemble method with *bagging* = bootstrap aggregating, where individual trees can be built in parallel.
- ▶ Nice illustration:
<https://mlu-explain.github.io/random-forest/>

Boosting

- ▶ Boosting is another ensemble method, by training classifiers *sequentially*.
- ▶ Each classifier focusses on correcting mistakes made by previous classifiers, gradually turning a weak classifier into a strong classifier.



Boosting

- ▶ Training data is weighted: misclassified instances get higher weights
- ▶ The voting will typically use a *weighted* average
- ▶ Examples: Adaboost, Gradient Boost, XGBoost
- ▶ Note: boosting can also be used in regression

Model evaluation: performance measures

Multiclass focus:

- **Classification accuracy:**

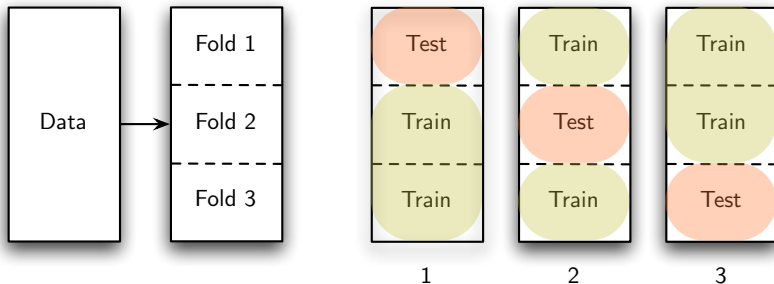
$$\frac{TP + TN}{TP + TN + FP + FN}$$

Single-class focus:

- Recall: $\frac{TP}{TP+FN}$
- Precision: $\frac{TP}{TP+FP}$
- **F measure:** $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ (used on unbalanced dataset Dance-9)

true \ pred.	P	N
P	TP true positive	FN false negative
N	FP false positive	TN true negative

Model evaluation: cross validation



- ▶ Data is subdivided into k cross-validation folds (typically $k = 5$ or $k = 10$, or *leave-one-out* cross validation)
- ▶ Model is trained using $k - 1$ folds and tested on the remaining fold
- ▶ Process is repeated k times, prediction errors are averaged

Feature selection

Feature selection is a ML technique designed to reduce the dimensionality of data, which is applied as a preprocessing step before the actual learning phase.

Theoretical properties:

- ▶ Aims to discover which features are highly correlated with the target class, and which ones are irrelevant
- ▶ Can lead to a compact feature set describing the data in a more interpretable, summarized way
- ▶ Decreases the size of the hypothesis space, and thereby leads to a faster and more effective search for the learning algorithms
- ▶ Tends to avoid overfitting
- ▶ Can lead to improved classification accuracies

Feature selection

Depending on the interaction between the feature selection technique and the classifier, we distinguish three types of methods:

- ▶ *Filter techniques*: computing predictive ability of individual features and filtering out the least relevant ones, i.e. independently of the classifier
- ▶ *Wrapper methods*: feature subsets are evaluated by the classifier with an internal cross-validation scheme, use of heuristics is mandatory
- ▶ *Embedded techniques*: optimal feature subset is constructed *during* the learning process

Feature selection and cross validation

Call $\mathcal{P}$ the following process:

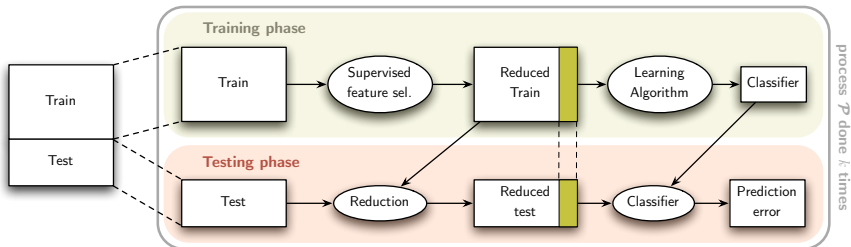
- ▶ Apply feature selection to the training set
- ▶ Apply the learning algorithm to the training set
- ▶ Reduce the test set, retaining only selected features
- ▶ Evaluate the resulting model on the reduced test set

In a cross-validation scheme, this means:

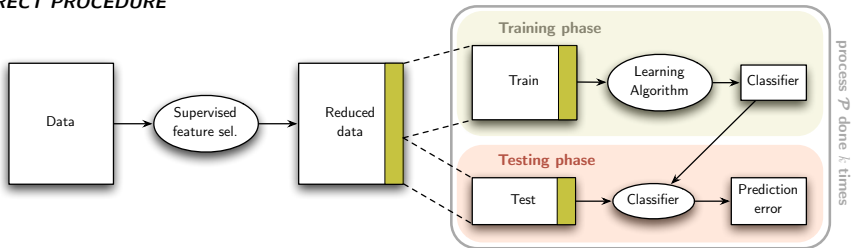
- ▶ Split the data in k folds
- ▶ Retain one fold for testing, the remaining $k - 1$ folds form the training set. Perform the entire process $\mathcal{P}$. Repeat this for every one of the k folds.

Feature selection and cross validation

CORRECT PROCEDURE



INCORRECT PROCEDURE



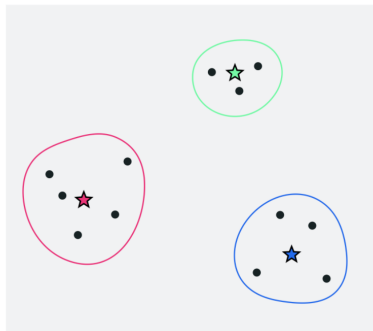
5. Unsupervised Learning

Unsupervised Learning

- ▶ Training data is a set of N input vectors $\mathbf{x}_i$ without any target labels.
- ▶ Discovering groups of similar examples is called *clustering*, e.g. K -means clustering or Hierarchical clustering.
- ▶ Determining the distribution of the data in the input space is called *density estimation*.
- ▶ Projecting the data from a high-dimensional space is called *visualization* or *dimensionality reduction*, e.g. Principal Component Analysis (PCA).

K-means clustering

- The goal is to discover K clusters, e.g. $K = 3$:



K-means clustering algorithm

Step by step:

- ▶ Choose K , the number of clusters
- ▶ Select K random centres μ_k ($k = 1, \dots, K$)
- ▶ Assign each data point $\mathbf{x}_i$ to the “closest” centre, creating K clusters
- ▶ Update the centre μ_k of each cluster
- ▶ Reassign each point $\mathbf{x}_i$ to the new closest centre
- ▶ Repeat the two previous steps until convergence

K-means clustering

Some questions/issues:

- ▶ Requires a distance or similarity measure!
- ▶ Importance of the choice of the initial centres? (empty clusters, sub-optimal clusters)
- ▶ Importance of the choice of K ? (what about overfitting?)
- ▶ Learning is minimizing the sum of squared distances of each point $\mathbf{x}_i$ to its cluster centroid $\boldsymbol{\mu}_k$:

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \text{ in Cluster } k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

Hierarchical clustering

- ▶ Identical goal as K -means clustering, but a different approach
- ▶ Two types: *agglomerative* (bottom-up) and *divisive* (top-down)

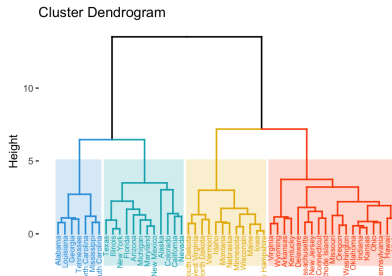
Steps in the agglomerative algorithm:

- ▶ At first, every point $\mathbf{x}_i$ is considered as a separate cluster
- ▶ Take the two closest input points to form one cluster
- ▶ Take the two “closest clusters” to form one cluster
- ▶ Repeat the previous step until there is only one cluster left

Hierarchical clustering (agglomerative)

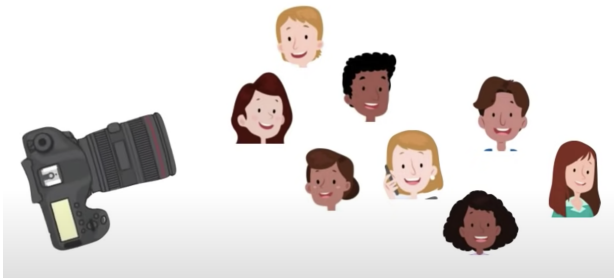
Some questions/issues:

- Requires a distance or similarity measure! (again!)
- What is meant by the distance between two clusters?? (closest - furthest - average - distance between centres)
- How to choose the final clusters? Representation by a *dendrogram*.



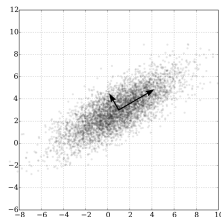
Principal component analysis

- ▶ Most well-known technique for data *visualization* and *dimensionality reduction*
- ▶ Core idea: find the “best direction” to take a picture of a group of people



Principal component analysis

- ▶ Back to the fish example: certain features are correlated, so maybe we can “blend” them to make a single feature without losing too much information?
- ▶ *Principal components* are optimal “blends” of features, i.e. linear combinations of features.
- ▶ Principal components are always orthogonal to each other.
- ▶ Principal components are ranked with respect to the amount of variation of the data they account for.



Principal component analysis

Some necessary mathematical ingredients to accomplish this:

► Mean: $\bar{\mathbf{x}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$

► Variance: in 1-dim: $\text{Var}(X) = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})^2$

► Covariance: $\text{Cov}(X, Y) = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(y_n - \bar{y})$

► Covariance matrix: in 2D:

$$S = \begin{pmatrix} \text{Var}(X) & \text{Cov}(X, Y) \\ \text{Cov}(X, Y) & \text{Var}(Y) \end{pmatrix} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T$$

Principal component analysis

Step by step:

- ▶ Shift data to put the centre at the origin.
- ▶ Compute the covariance matrix S .
- ▶ Compute the eigenvectors and eigenvalues of S (which is a symmetric matrix!).
- ▶ Only keep the k eigenvectors corresponding to the k largest eigenvalues. (Typically: $k = 1, 2$)
- ▶ Project the data on those k eigen-directions.

Principal component analysis

For example:

- ▶ if $k = 1$, then all the data points are projected on the line determined by PC1.
- ▶ If $k = 2$, then all the is projected on the plane formed by PC1 and PC2.

Check this visualization:

<https://setosa.io/ev/principal-component-analysis/>

Principal component analysis

Learning in PCA:

- ▶ Projection of a data point $\mathbf{x}_n$ on a unit vector $\mathbf{u}_1$ is the scalar $\mathbf{u}_1^T \mathbf{x}_n$
- ▶ The variance of the projected data is:

$$\begin{aligned} \frac{1}{N} \sum_{n=1}^N \left(\mathbf{u}_1^T \mathbf{x}_n - \mathbf{u}_1^T \bar{\mathbf{x}} \right)^2 &= \frac{1}{N} \sum_{n=1}^N \mathbf{u}_1^T (\mathbf{x}_n - \bar{\mathbf{x}}) (\mathbf{x}_n - \bar{\mathbf{x}})^T \mathbf{u}_1 \\ &= \mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 \end{aligned}$$

So, when $\mathbf{u}_1$ is an eigenvector relatif to eigenvalue λ_1 , this variance - which we want to be maximal - is:

$$\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1 = \mathbf{u}_1^T \lambda_1 \mathbf{u}_1 = \lambda_1$$

Therefore, PC1 is the eigenvector corresponding to the largest eigenvalue λ_1 .

6. Reinforcement Learning

Stating the reinforcement problem

Key elements in reinforcement learning:

- ▶ Learning from interaction: an active decision-making *agent* can undertake certain *actions* in an uncertain environment, and gets a *reward* upon desired outcomes and a *penalty* upon undesired outcomes.
- ▶ Examples: learn to play chess, a robot learns how to walk or ride a bike, etc. Historical example: the multi-armed bandit problem.
- ▶ The agents actions influence its future inputs, over an extended time period.
- ▶ The agent has no direct instructions on which actions to take.
- ▶ Trade-off between *exploration* and *exploitation*.
- ▶ Learning = maximize the reward signal.

Stating the reinforcement problem

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ." [Mitchell (1997)]

Reinforcement learning:

- ▶ Experience E : the current and previous states, the past rewards and penalties
- ▶ Task T : a specific goal in an uncertain environment
- ▶ Performance measure P : a reward signal

An example: tic-tac-toe

An reinforcement learning approach for tic-tac-toe:

- ▶ Set up a table containing a *value* for each *state* of the game: this value is the latest estimate of the probability of winning from that state.
- ▶ The table is acting like a value function that is continuously updated/learned while playing.
- ▶ Two types of moves: *greedy* and *exploratory*.
- ▶ After a greedy move, the current value $V(s)$ of the earlier state s is updated:

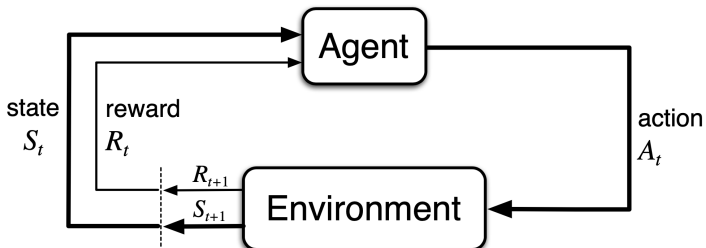
$$V(s) \leftarrow V(s) + \alpha(V(s') - V(s))$$

where s' is the state after the greedy move, and α is the learning rate.

- ▶ The value function converges to an optimal playing *policy*.

Reinforcement learning

- ▶ The interaction between the agent and its environment:



- ▶ The *policy* π_t is the mapping from states to probabilities: $\pi_t(a|s)$ is the probability that $A_t = a$ if $S_t = s$.
- ▶ RL methods specify how the agent updates its policy as a result of its experience.

Reinforcement learning: finite Markov decision processes

- ▶ A state signal has the *Markov property* if S_{t+1} and R_{t+1} only depend on the previous state S_t and action A_t . So:

$$P(R_{t+1} = r, S_{t+1} = s' \mid S_0, A_0, R_1, \dots, S_{t-1}, A_{t-1}, R_t, S_t, A_t)$$

is equal to

$$P(R_{t+1} = r, S_{t+1} = s' \mid S_t, A_t)$$

- ▶ Moreover, if the state and action spaces are finite, the reinforcement learning task is called a *finite Markov decision process*.

7. Sequential data

Sequential data

- ▶ Typical examples are: DNA sequences, weather forecast, currency exchange rates, time frames in speech recognition, characters in a sentence, notes in musical piece, etc.
- ▶ Typical models are: *Markov models* and *Hidden Markov models* (amongst others)

Markov models

- ▶ Product rule: $p(\mathbf{x}_1, \dots, \mathbf{x}_N) = \prod_{n=1}^N p(\mathbf{x}_n | \mathbf{x}_1, \dots, \mathbf{x}_{n-1})$
- ▶ *First-order Markov chain*: assumption that the probability of an observation only depends on the previous one:

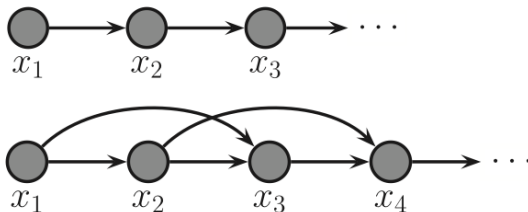
$$p(\mathbf{x}_1, \dots, \mathbf{x}_N) = p(\mathbf{x}_1) \prod_{n=2}^N p(\mathbf{x}_n | \mathbf{x}_{n-1})$$

- ▶ *Second-order Markov chain*: assumption that the probability of an observation only depends on the two previous ones:

$$p(\mathbf{x}_1, \dots, \mathbf{x}_N) = p(\mathbf{x}_1)p(\mathbf{x}_2 | \mathbf{x}_1) \prod_{n=3}^N p(\mathbf{x}_n | \mathbf{x}_{n-2}, \mathbf{x}_{n-1})$$

Markov models

- ▶ Markov models illustrated:



- ▶ Roughly speaking, building a Markov model comes down to counting the frequencies of all sequences occurring in the training data

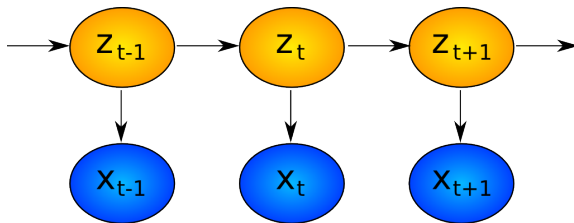
Classification with Markov models

- ▶ For every class $\mathcal{C}_k$ a separate model is constructed
- ▶ A test sequence $\mathbf{x}_1, \dots, \mathbf{x}_N$ is classified according to the model that “generates” it with the highest probability, i.e. we determine the class $\mathcal{C}_k$ that maximizes

$$p(\mathbf{x}_1, \dots, \mathbf{x}_N | \mathcal{C}_k)$$

Hidden Markov models

- Addition of discrete latent variables



8. Sustainable AI: environmental impact

Sustainable IT - Numérique Responsable

- ▶ IT has 4 types of impact:
 - ▶ Environmental
 - ▶ Social
 - ▶ Ethical
 - ▶ Security
- ▶ Our focus is on reducing the *environmental* impact, and how to use IT as a meaningful tool in the ecological transition.
- ▶ Interesting initiatives and ressources:
 - ▶ ISIT
 - ▶ paradigm.brussels (membre de ISIT)

Issues

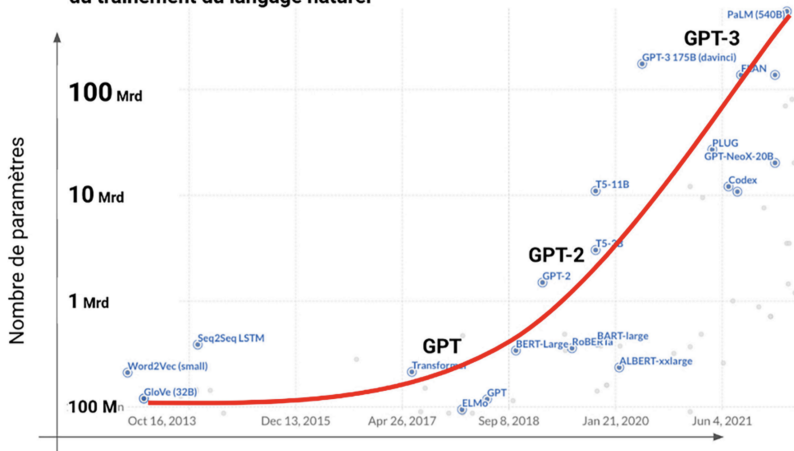
- ▶ Increasing amounts of data exchange and storage
- ▶ Increasing number and size of data centers
- ▶ Increasing number of numeric equipments

AI applications are only accelerating this more!

- ▶ ChatGPT quest uses 10 times more electricity than on Google
- ▶ Image generation 100 times more electricity than text classification
- ▶ Number of users using generative AI is increasing rapidly
- ▶ AI requires even bigger data centers, whose electricity and water consumption are exploding. Projection: in 2050, their electricity consumption will exceed current consumption of the entire European Union. AI applications will account for 68% of that, compared to 11% today.

Model size evolution

Évolution de la taille des modèles dans le domaine du training du langage naturel



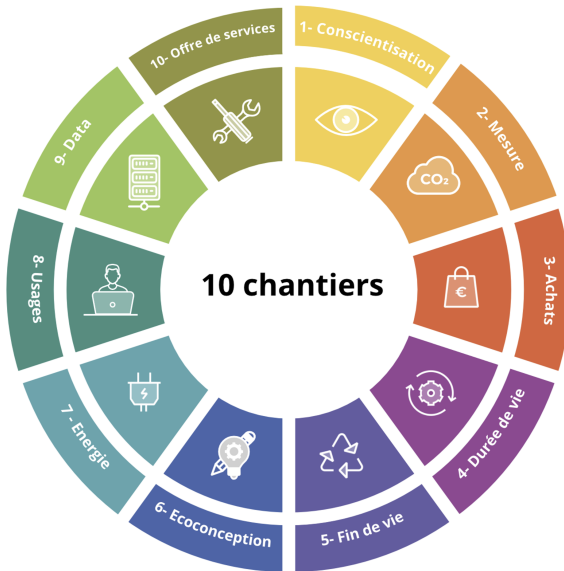
Mesuring the environmental footprint

- ▶ Carbon emission
- ▶ Non-renewable energy consumption (mainly oil)
- ▶ Decrease of ressources, e.g. metals (mining industry)
- ▶ Emission of chemical products
- ▶ Water consumption (mines, data centers)
- ▶ Water and soil pollution
- ▶ Biodiversity loss

The numerical footprint is mainly caused by the manufacturing of portable computers, smartphones, televisions, etc.

- ▶ Example of a tool to measure: EcoLogits calculator

Solutions



Source: paradigm.brussels

9. Bibliography

Bibliography

- C. M. Bishop. *Pattern recognition and machine learning*, volume 4. Springer New York, USA, 2006.
- N. Japkowicz and M. Shah. *Evaluating Learning Algorithms: A Classification Perspective*. Cambridge University Press, 2011.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.
- T. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997.
- V. N. Vapnik. *The nature of statistical learning theory*. Springer-Verlag New York Inc, 1995.